

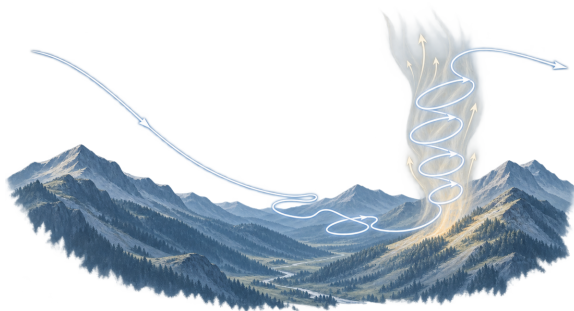
INTERNSHIP & THESIS ROADMAP

From flight logs to optimal paths

Possible research paths
and timeline

Matteo Di Sante

September 2026 – February 2027



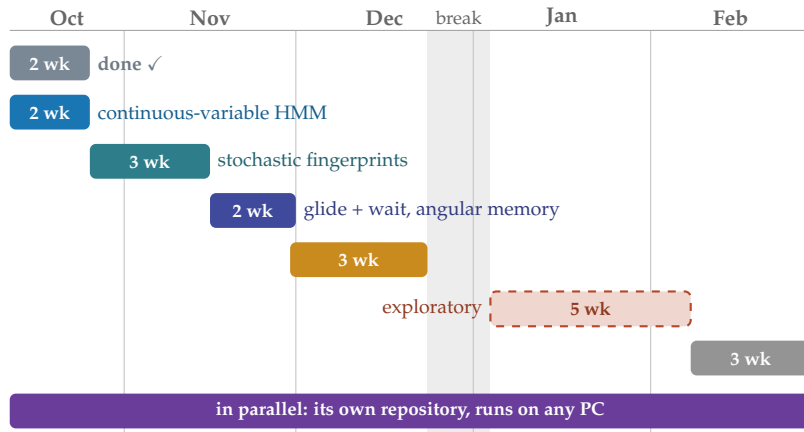
Research roadmap | Slide 1

Today: the possible paths for the rest of the internship and the thesis, from October 2026 to February 2027. Six steps. The first is done. Steps 2 and 3 are the foundation. Steps 4, 5 and 6 are research directions built on top of it. A graphical tool is developed in parallel throughout. Cover: AI-generated illustration of a glide, a search and a climb in a thermal.

The plan at a glance

12 Oct 2026 – 28 Feb 2027 2

- 1 Data**
done; simplify 12–25 Oct
 - 2 Segmentation**
12–25 Oct
 - 3 Characterisation**
26 Oct – 15 Nov
 - 4 Stochastic model**
16–30 Nov
 - 5 Solo vs group**
30 Nov – 23 Dec
 - 6 Optimal path**
4 Jan – 7 Feb
- Buffer**
7–28 Feb
- Graphical tool**



Buffer: absorb delays, or go deeper on the most interesting results

Research roadmap | Slide 2

Six steps. Step 1, data download and preprocessing, is done. Today's preprocessing has too many parameters, is too complex and discards too many flights; I will simplify it in parallel with segmentation, 12–25 October. Segmentation (step 2) and the stochastic characterisation (step 3) are the foundation: the phase labels and the measurement toolbox used by everything that follows. Then three research directions: a stochastic model (step 4), solo versus group flight (step 5), and an exploratory optimal-path study (step 6, dashed because its scope is still open).

Indicative windows: segmentation 12–25 October (2 weeks), characterisation 26 October – 15 November (3 weeks), stochastic model 16–30 November (2 weeks), solo vs group 30 November – 23 December (3 weeks), winter break 24 December – 3 January, optimal path 4 January – 7 February 2027 (5 weeks), then three weeks of buffer until 28 February. The buffer absorbs delays, or is spent on whatever turns out most interesting: parts of the solo vs group study, of the stochastic model, or of the optimal path. The graphical tool runs in parallel for the whole period. Bars are drawn to scale in calendar days.

Download

186,052

paraglider tracklogs

6,716

hang-glider tracklogs

preprocessing
→

Clean archive

156,406

paraglider flights (84.1%)

6,094

hang-glider flights (90.7%)

Simplify the preprocessing

NEXT

Today's version has too many parameters, is too complex and discards too many flights (paragliders: 16%).
In parallel with segmentation, 12–25 Oct.

The FFVL distance-contest archive is downloaded: 186,052 paraglider and 6,716 hang-glider tracklogs. The preprocessing pipeline cleans the recorded flight, selects flights and projects them to local coordinates, puts them on a uniform time grid and smooths them with a Savitzky–Golay filter. It keeps 156,406 paraglider flights (84.1%) and 6,094 hang-glider flights (90.7%). It works, but it has too many parameters, it is too complex, and it discards too many flights for no good reason: today paragliders lose 16% of the downloaded tracklogs. I will simplify it in parallel with the segmentation work, 12–25 October.

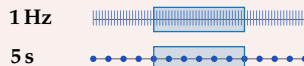
Vilpellet's HMM

 X^{v_z} : climbing? X^{curv} : turning one way? X^{str} : straight?

yes/no, on a rolling 30 s window

Checked in the viewer:
works well, but misses
some climbs

Only for 1 Hz



30%

paragliders

72%

hang gliders

of flights left out: loggers
slower than 1 Hz

Continuous HMM



vertical speed

turn rate

turn coherence

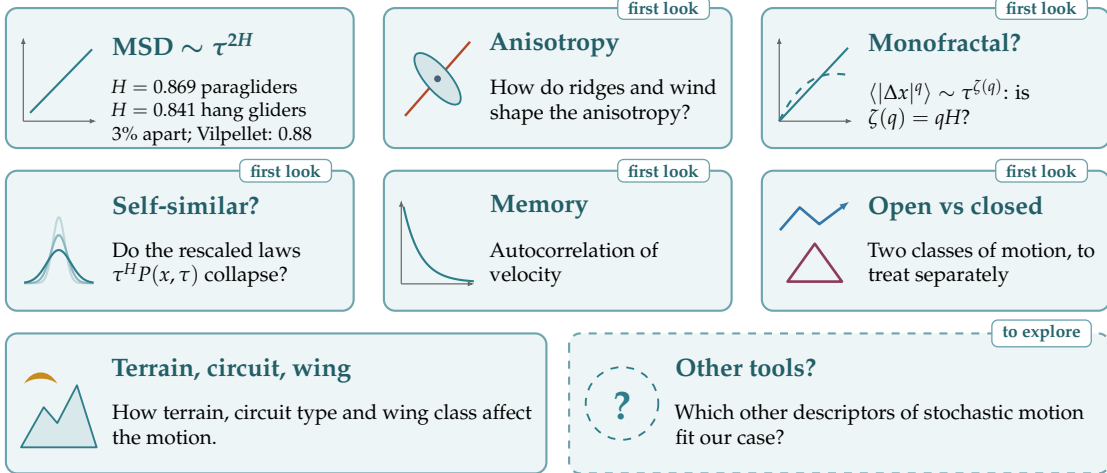
$$\begin{aligned} &\langle \dot{z} \rangle_W \\ &\langle |\dot{\theta}| \rangle_W \\ &\frac{|\int_W \dot{\theta} dt|}{\int_W |\dot{\theta}| dt} \end{aligned}$$

W: time window

To validate:
downsampled 1 Hz flights
and/or hand-labelled
flights

Vilpellet's HMM observes $X_t = (X_t^{v_z}, X_t^{\text{curv}}, X_t^{\text{str}})$, three binary features computed over a rolling 30 s window, about the period of one full turn in a thermal. $X_t^{v_z} = 1$ if the mean vertical speed over the window is positive. $X_t^{\text{curv}} = 1$ if the sign of the curvature angle is preserved for at least 90% of the samples in the window. $X_t^{\text{str}} = 1$ if the absolute curvature angle stays below a threshold for at least 90% of the window; the threshold comes from the distribution of curvature angles: 0.21, 0.17 and 0.10 rad for paragliders, hang gliders and sailplanes. The phase sequence is then decoded with the Viterbi algorithm. Looking at many flights in the viewer, it works well but misses some climb segments, and climbs matter for the whole project. It was fitted on 1 Hz tracks: on slower loggers the same window holds few fixes (6 at 5 s) and the features become noisy, so 30.1% of paraglider and 72.0% of hang-glider flights are left out. Next: an HMM with continuous variables, vertical speed v_z , turn rate $|\dot{\theta}|$ and turn coherence $|\int \dot{\theta} dt| / \int |\dot{\theta}| dt$ (1 when circling, 0 when left and right turns cancel), validated on 1 Hz flights downsampled to 2–10 s and on hand-labelled flights.

Goal: phase labels for all flight time, whatever the logger

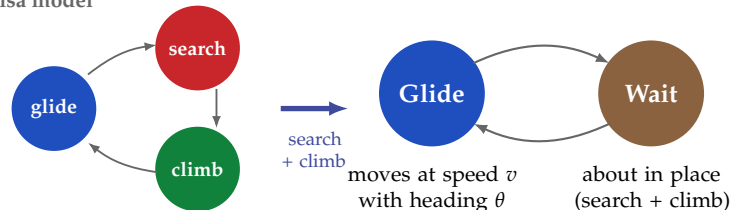


So far the stochastic description rests on the MSD and H . Different processes can share the same MSD. A broader characterisation is needed both to describe the motion and to test which properties a stochastic model reproduces.

The items marked “first look” have preliminary results, in the thesis or at the offsite, to consolidate. MSD and H : our preliminary analyses give $H = 0.869$ for paragliders and $H = 0.841$ for hang gliders, a 3% difference; Vilpellet et al. found $H \approx 0.88$ for all aircraft. Anisotropy: how mountain profiles and wind shape the motion. Moment scaling: is $\zeta(q)$ linear in q (monofractal) or concave (multifractal)? A linear $\zeta(q)$ over a finite range is evidence of monofractal scaling, not proof of self-similarity. Self-similarity: do displacement distributions collapse after rescaling by τ^H ? Memory: velocity and heading autocorrelations, feeding the model of step 4. Open vs closed circuits: probably two classes to analyse separately; glide autocorrelation already shows different behaviour after the first glides. Terrain, circuit type and wing class: their effect on all these properties, not only on H .

The last card: survey other descriptors in the literature (first-passage times, turning-angle statistics, power spectra, stationarity and ergodicity tests) and assess which are informative for our data. Compare whole flights and phases using the labels from step 2.

Pisa model



Memory decides long times

$$\text{MSD}(\tau) = 2 \int_0^\tau (\tau - s) C_v(s) ds$$

$\int_0^\infty C_v ds < \infty$: diffusive

$C_v \sim s^{-\gamma}, \gamma < 1$: $\text{MSD} \sim \tau^{2-\gamma}$

1 Heading data

$C_\theta(\tau)$ within and across glides

2 Angular law

an equation for $\theta(t)$ compatible with the observed data

3 Long times

asymptotic superdiffusion?

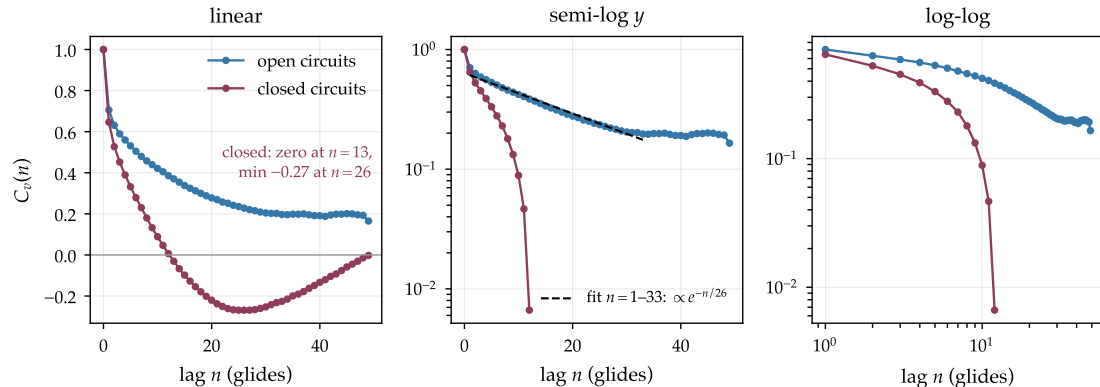
4 Check

not only H : the other features measured in step 3

Simplification of the Pisa model: two states only. Glide: the pilot moves at speed v along heading θ . Wait: search and climb merged; at coarse scale the horizontal position barely moves. This approximation must be checked against the empirical search and climb MSDs (Vilpellet et al., Fig. 3), adding a local drift if needed.

The core is the angular dynamics. First measure what the data say: heading autocorrelation $C_\theta(\tau) = \langle \cos[\theta(t + \tau) - \theta(t)] \rangle$ within a glide, and the correlation between the headings of successive glides across a wait. Then formulate an angular diffusion consistent with it. Then compute the long-time behaviour: for a stationary velocity, $\text{MSD}(\tau) = 2 \int_0^\tau (\tau - s) C_v(s) ds$. If C_v is integrable the motion becomes diffusive at long times; ordinary Brownian angular diffusion gives exponential decay, so only a transient superdiffusion. A power-law tail $C_v \sim s^{-\gamma}$ with $0 < \gamma < 1$ gives $\text{MSD} \sim \tau^{2-\gamma}$. C_v here is the full velocity correlation, including the glide/wait switching. Finally compare the model's statistics with the fingerprints of step 3, not only with H .

$\mathbf{v}_k = \Delta \mathbf{r}_k / T_k$: mean velocity vector of glide k . $C_v(n) = \langle \mathbf{v}_k \cdot \mathbf{v}_{k+n} \rangle / \langle |\mathbf{v}_k|^2 \rangle$



Open vs closed: very different motion from the first lags.
Open: $C_v \approx 0.63 e^{-n/26}$ up to $n = 33$: diffusive at long times?

Paragliders, fixed cohort: flights with at least 50 glides in one segment (963 open, 2,407 closed), each weighted equally.

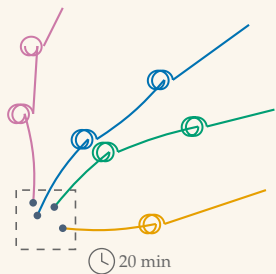
First look at the data behind the model of the previous slide. Each Vilpillet glide (transition run) gets one mean velocity vector, $\mathbf{v}_k = \Delta \mathbf{r}_k / T_k$: the horizontal displacement vector from the start to the end of the glide, divided by its duration. Glides are numbered in time order inside a cleaned segment.

Fixed cohort: only flights with at least 50 glides in one segment enter (963 open and 2,407 closed circuits, paragliders), so every flight contributes to every lag up to $n = 49$ and the population does not change with the lag. For each flight, $\mathbf{v}_k \cdot \mathbf{v}_{k+n}$ is averaged over all its pairs n glides apart; these flight averages are then averaged with equal weight and divided by the mean squared glide speed. C_v is not centred: a common route direction shows up as a positive value at long lags.

Open circuits: $C_v(1) = 0.71$, slow monotone decay, still 0.16 at $n = 49$. A straight line in semi-log y over $n = 1-33$ gives $C_v \approx 0.63 e^{-n/26}$: a first hint of exponential memory loss, hence of diffusive motion at long times. Only a first check: the decay slows down with the lag and the curve flattens near 0.16; a positive plateau would be a persistent route direction, i.e. a ballistic drift. Next check: remove each flight's mean velocity. Closed circuits: $C_v(1) = 0.65$, zero at $n = 13$, minimum -0.27 at $n = 26$, back to about zero at $n = 49$: to close the circuit the pilot must fly back.

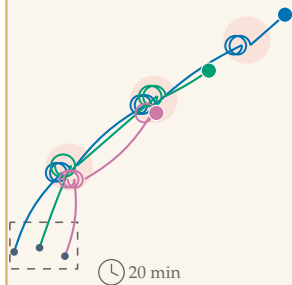
Consequence for the model: the two circuit types already differ after the first glides, as they do for H , so they should be treated as two separate classes of motion. Angular diffusion alone cannot produce either the persistent direction of open circuits or the anticorrelation of closed ones; treat the two separately or remove each flight's mean direction. No error bars yet.

Same take-off
cell, same 20 min



same start, different paths

Flying together



same thermals

1 An operational definition

Same take-off cell +
same departure window
does not imply group flight



2 Then the analysis

solo vs group: where they agree
and where they differ

Then compare with step 3:

MSD and H

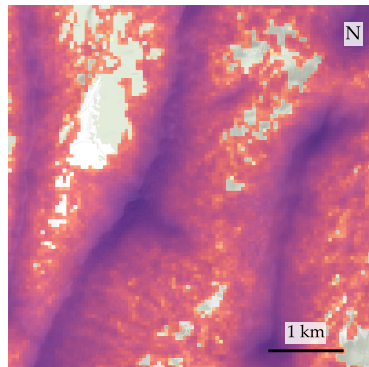
scaling

memory

phase durations

Two steps. First, an operational definition of group flight: both sketches show departures from a common cell within the same 20-minute window, but only the right-hand group follows the same thermals. A common departure window alone does not establish group flight. The current viewer lists only candidates: departures from the same 5 km cell within a 30-minute window. The first task is to find the definition that groups the real group flights, separating them from flights that simply share a site, a day or a route; weather and common routes alone can create correlated motion.

Second, the comparison: once the labels are reliable, apply the toolbox of step 3 to solo and group flights (MSD and H , scaling, memory, phase durations) to see where they are similar and where they differ. Uncertainty must be computed at the level of site and day or of the group, since group members are not independent.



10⁻² 10⁰ 10² 10⁴
Thermal hours/km² (log)

Chartreuse, 5 × 5 km.
All years pooled.

- 1 **Data on glider climb segments**
→ a thermal distribution $P(x, y)$

Several definitions, depending on the research question.

- 2 Updraft distribution $p(w \mid \text{site})$
 w : vertical air velocity

How do both distributions change?
between terrain classes: mountains, hills, plains
within one class: from one mountain (or plain) to another

- 3 Optimal 2D path for each terrain type,
using the thermal distributions from the data

- 4 Compare optimal paths with recorded flights

Compute an optimal trajectory from empirical thermal distributions and compare it with recorded flights. Almgren and Tourin (2015) use Hamilton–Jacobi–Bellman equations; I have not studied the paper yet. A 2D extension is exploratory.

Steps 1 and 2 estimate $P(x, y)$ from glider climb segments and the updraft distribution $p(w \mid \text{site})$. Define what $P(x, y)$ counts or weights, and its normalisation, according to the research question. The map shows cumulative climb time per area, not encounter probability. Here w is vertical air velocity; observed climb rate is $\dot{z} = w - s_{\text{sink}}$, so inferring w requires a sink-rate model.

Compare both distributions across mountains, hills and plains, then across sites within each category. Test whether sites are compatible with a common distribution: a shared law is a hypothesis. Comparing spatial distributions also requires a meaningful common coordinate representation.

Step 3 uses these empirical distributions in a 2D control problem depending on terrain category. A shared distribution may provide a category-level model if supported by the data; otherwise retain site differences. The loss function and formulation remain to be defined. Step 4 compares optimal and recorded paths.

Sources: R. Almgren and A. Tourin, Optim. Control Appl. Meth. (2015), doi:10.1002/oca.2122. Map: viewer Thermal density, Vilpelle, paragliders and hang gliders, all dates and heights; 50 m pixels, hours/km², fixed logarithmic scale. Background: IGN/Géoplateforme, Plan IGN.



Today

Works, but **not reviewed or tested**

Mixed with the analysis code, so it **runs only on my Mac**

Short term: today's viewer

- own repository, runs on any PC
- no new work on it

Usable soon, still unreviewed

In parallel: a clean viewer

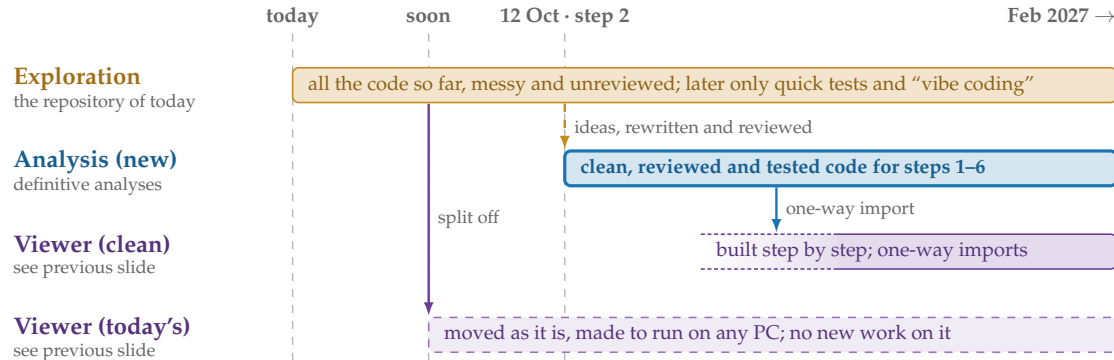
- reviewed, one-way imports, any PC
- all new features go here

Trustworthy, but it takes longer

Make today's viewer usable by everyone, but invest the time only in the clean viewer

The viewer was built with Claude, on top of analysis code that has not been reviewed. It works, and it has been very useful, but it has not been properly reviewed and tested, so we cannot be sure that everything it shows is computed as it should be. Its code is mixed with the analysis code of the repository that holds everything written so far, with imports in both directions: this is the main reason why today it hardly runs on any computer other than mine.

Two things can be done, with different roles. In the short term, move today's viewer, unreviewed, to a separate repository and spend some time making it run on other computers. It is a temporary solution, but it lets everyone use the current viewer soon, knowing it is not reviewed; it should not do much harm. In parallel, and over a longer time, build a clean viewer: reviewed code, which imports the tools of the clean analysis repository in one direction only, designed from the start to run on any PC and to be easy to install and use. Both take time, which is why the tool runs as a parallel track for the whole internship. Because today's viewer is so entangled with the analysis code, it makes no sense to keep improving it: once it runs everywhere, it stays as it is. New features and fixes go into the clean viewer, so that the time is spent on code meant to be used by everyone, reviewed and kept in the long run.



Explore fast in one place; build the internship results only on reviewed code

Today there is one repository with everything: download, cleaning, experiments, analyses and the viewer. It was written with Claude to explore as much as possible, as fast as possible, and much of it has never been reviewed. That is fine for exploring, not for the results of the internship.

The plan over time. Soon, today's viewer is split off, as it is, into its own repository and made to run on any PC (previous slide); no new work goes into it. From 12 October, with step 2, a new repository with clean, reviewed and tested code holds the definitive analyses of each step. In parallel, step by step, a clean viewer is built on top of it, importing the analysis tools in one direction only (previous slide). The current repository stays for exploration, quick tests and “vibe coding”; ideas that work there are rewritten and reviewed in the clean one, never copied over as they are. Reviewing takes much more time than writing, but it is the only way to be sure that everything works as expected.